Built from your latest feedback · 7.8–8.3/10, moderate-good chance

Kubernetes Production Deep-Dive

 29 questions

7 sections

Precision, not breadth

Reviewed

0 / 29

Why this one's different from your other documents

This isn't new topics — it's the same six things that got flagged, answered with the exact precision the interviewer was pushing for. The strategy vs. technique distinction on deployments is the single most important thing in here — he corrected it multiple times, and it's a specific, checkable fact you can nail every time once it's locked in, not a matter of experience.

Coverage — tap a tile to jump

Deployment Strategies

0/7

K8s Networking — TLS

0/6

Autoscaling

0/4

Helm

0/3

Docker

0/4

SRE Concepts

0/3

Production Incident

0/2

Search questions & answers…

Collapse all

All

Deployment Strategies

K8s Networking — TLS

Autoscaling

Helm

Docker

SRE Concepts

Production Incident

§1 · CRITICAL

Deployment Strategies

7 Q&A

The biggest gap — the exact strategy vs. technique taxonomy, hit precisely.

01

What's the actual difference between a deployment STRATEGY and a rollout TECHNIQUE?

›

Blue-Green is a deployment strategy: you run two complete, independent environments (Blue = current, Green = new), fully test Green in isolation with zero customer traffic, then cut traffic over — either all at once or in one deliberate step — and Blue stays fully intact as an instant rollback target. Rolling and Canary are rollout techniques: they describe HOW you introduce the new version's traffic gradually within essentially one environment/mechanism, not a separate parallel environment. 'Strategy' answers 'do I have a full parallel environment to test in and cut over to.' 'Technique' answers 'how gradually do I expose the new version once I'm rolling it out.'

Say this distinction explicitly and early — it's exactly what got corrected multiple times

Blue-Green = two full separate environments, tested in isolation before any customer sees it

Rolling/Canary = gradual traffic introduction within one rollout mechanism, not a separate environment

Reviewed

02

Explain Blue-Green — why does it minimize downtime, and why do customers sometimes still see issues?

›

Downtime minimizes because the cutover — flipping a router/load-balancer/service selector from Blue to Green — is close to instantaneous, versus a slow rebuild-in-place. But 'minimizes' isn't 'eliminates': in-flight requests at the exact moment of cutover can drop if connection draining isn't handled properly, caching/CDN can serve stale content briefly, and if Green has a bug that only shows under real production traffic patterns, customers hit it immediately at 100% once cutover happens — there's no gradual exposure to catch it early, unlike canary.

Cutover is near-instant (router/selector flip) — that's WHY it minimizes downtime

'Minimizes' not 'eliminates' — connection draining, caching/CDN staleness, and a bug only visible under real traffic are the real gaps

A bug in Green hits 100% of traffic immediately with a hard cutover — no gradual exposure, unlike canary — name this tradeoff

Reviewed

03

How do you test before cutover in Blue-Green, and how does rollback actually work?

›

Before cutover: smoke tests and synthetic traffic against Green directly via its own internal endpoint, not the public route yet, plus health checks confirming all pods are Ready. Rollback is the mirror of cutover: flip the router/selector back to Blue, which is still fully running and untouched — this is why Blue-Green rollback is fast, seconds, not a redeploy, since nothing is being rebuilt, just repointed.

Test against Green's internal endpoint before it's public-facing — pre-cutover validation, not post-cutover discovery

Rollback = flip back to Blue, which never stopped running — that's WHY it's fast, not a redeploy

Keeping Blue alive (not torn down) until confident in Green is the whole point

Reviewed

04

Canary deployment — walk through traffic shifting from one pod to full rollout.

›

Start by routing a small percentage of real traffic, often literally one pod out of ten (roughly 10%), to the new version while the rest still serve the old one, watching real production metrics against that slice specifically, not aggregate. If healthy, increase in steps (10% → 25% → 50% → 100%), pausing to observe at each step; if unhealthy, abort and route back to 0%. Customer impact is bounded to whatever percentage was live at the point you caught and aborted it — fundamentally different from Blue-Green's all-or-nothing exposure.

One pod out of ten IS roughly 10% canary — connect the concrete mechanic to the percentage concept

Watch metrics on the canary slice specifically, not blended aggregate numbers that dilute a real problem

Bounded blast radius is the core value proposition versus Blue-Green's all-or-nothing

Reviewed

05

Blue-Green vs Canary — the sharpest possible direct comparison.

›

Blue-Green trades capacity (double the infrastructure temporarily) for a fast, clean, all-or-nothing cutover with instant, zero-rebuild rollback. Canary trades a slower, gradual rollout for a bounded blast radius — a bug only ever reaches a fraction of traffic before you catch and abort it, at the cost of running two versions simultaneously longer and needing good metrics to judge each step. Neither is strictly better; Blue-Green suits changes you're fairly confident about, canary suits changes where you want production traffic itself part of the validation.

Blue-Green: capacity cost, fast clean cutover, instant rollback, all-or-nothing exposure

Canary: time cost (gradual), bounded blast radius, needs good real-time metrics to judge each step

Framing it as a tradeoff rather than 'canary is just better' shows senior-level judgment

Reviewed

06

Rolling update — how is it different from both, and what does Kubernetes do natively?

›

A rolling update is Kubernetes' built-in default Deployment strategy: it replaces old pods with new ones incrementally, controlled by maxSurge (extra pods above desired count it can create) and maxUnavailable (how many can be down at once), with no separate parallel environment and no traffic-percentage control the way canary has. It's the default because it needs no extra infrastructure, but offers less control than canary and less safety than Blue-Green.

maxSurge/maxUnavailable are the two knobs — name them specifically, a concrete checkable detail

It replaces pods incrementally — different from canary's traffic-percentage control on otherwise-unchanged pod counts

It's the K8s default because it needs zero extra infrastructure — that's the actual reason, not that it's 'better'

Reviewed

07

kubectl rollout undo vs Helm rollback — when do you use each?

›

kubectl rollout undo deployment/name reverts to the previous ReplicaSet already recorded in the Deployment's revision history — fast, no extra tooling, but limited by revisionHistoryLimit and only rolls back the Deployment object itself. helm rollback release-name revision-number reverts the entire release — every resource Helm manages, not just one Deployment — which matters when a change touched a ConfigMap, Service, and Deployment together and they need to revert as a consistent

set.

kubectl rollout undo = one Deployment object, from K8s's own revision history

helm rollback = the whole release (every resource the chart manages), from Helm's release history

If a change touched multiple resource types together, Helm rollback is the consistent, correct tool

Reviewed

§2 · CRITICAL

K8s Networking — TLS

6 Q&A

cert-manager, mTLS, service mesh, the exact API-A/API-B scenario.

01

API A can't communicate with API B — TLS handshake failure. Full RCA process.

›

Start with openssl s_client -connect service-b:443 from inside API A's pod to see the actual handshake failure and certificate presented. Then work through, in order: certificate expiry (check NotAfter — the single most common cause), CA chain trust (does A's trust store include the CA that signed B's cert), whether the Secret backing the cert is stale or was rotated but not reloaded by the pod, and time synchronization (a clock-skewed node rejects a genuinely valid cert). If cert-manager issues the cert, check its logs and the Certificate resource's status for a renewal that silently failed.

openssl s_client from inside the source pod is the direct diagnostic — start there, not with theory

Order: cert expiry → CA trust chain → stale Secret not reloaded → time sync → cert-manager renewal failure

This is close to verbatim what was asked — have this sequence ready cold

Reviewed

02

What does cert-manager actually do, and how do you troubleshoot it?

›

cert-manager automates certificate issuance and renewal: you define a Certificate resource, it requests one from an Issuer/ClusterIssuer, and writes the result into a Kubernetes Secret your Ingress or pod mounts — then it watches expiry and renews before it, rewriting that same Secret. When something's wrong: kubectl describe certificate shows current status (a stuck 'False' Ready condition first), then describe certificaterequest and order/challenge (for ACME issuers) show exactly where issuance is stuck — often a failed HTTP-01 or DNS-01 challenge from a networking/DNS misconfiguration.

Certificate → Issuer/ClusterIssuer → Secret is the object chain — know it in that order

describe certificate first for overall status, then drill into certificaterequest/order/challenge

A failed ACME challenge due to networking is the most common real-world cert-manager failure

Reviewed

03

How do Kubernetes Secrets relate to TLS, and what's a common failure mode?

›

A TLS Secret holds the cert and private key (tls.crt/tls.key) that an Ingress or pod mounts to terminate or present TLS. The common failure: cert-manager rotates the Secret's contents on renewal, but a pod that mounted it as a volume rather than reading it fresh doesn't automatically pick up the new cert — some apps need a restart or reload signal, so 'the Secret updated fine but the app is still presenting the expired cert' is a real, distinct failure mode.

TLS Secret = tls.crt + tls.key, mounted by Ingress or pod

Secret rotating does NOT guarantee the consuming pod picks up the change automatically — that's the trap

Distinguish 'Secret is wrong' from 'Secret is right but the app hasn't reloaded it'

Reviewed

04

What's mTLS, and why use it over regular TLS between internal services?

›

Regular TLS authenticates the server to the client but not the reverse. Mutual TLS adds client authentication: both sides present and verify certificates, so API B also confirms API A is who it claims before accepting the connection. Inside a cluster, a compromised pod that gets network access to API B shouldn't be able to just talk to it — mTLS means it also needs a valid client certificate, which it won't have. A service mesh is the common way to get mTLS everywhere without hand-rolling certificate management per service.

Regular TLS = server proves identity to client. mTLS = both sides prove identity to each other

Inside a cluster, mTLS stops network-level access alone from being enough to reach a service

Naming a service mesh as the practical way to get mTLS everywhere is the senior-level follow-up

Reviewed

05

What does a service mesh add specifically for TLS/networking?

›

A service mesh injects a sidecar proxy next to every pod that transparently handles mTLS between every service with zero application code changes — the app makes a plain HTTP call, the sidecar handles TLS underneath. It also gives fine-grained traffic policy (which services can talk to which), automatic cert rotation for the mesh's own internal certs, and consistent mTLS telemetry so a handshake failure shows up centrally rather than only in one app's logs.

Sidecar handles mTLS transparently — app code stays plain HTTP

Automatic cert rotation for internal service-to-service certs, separate from cert-manager's Ingress-facing certs

Mesh-level telemetry means a TLS failure is visible centrally, not buried in one app's logs

Reviewed

06

How do internal DNS and ClusterIP factor into a service-to-service TLS scenario?

›

Before TLS happens, API A has to resolve API B's ClusterIP via CoreDNS, then connect to it — if either fails, you'd see a connection failure or timeout, not a TLS handshake failure specifically. That's a useful diagnostic shortcut: getting as far as a TLS failure (rather than a timeout) already confirms DNS and basic connectivity were fine, narrowing the problem to the certificate/trust layer.

DNS resolution and ClusterIP connectivity happen BEFORE TLS — a TLS-specific failure means those already succeeded

Use this as a diagnostic shortcut: reaching a TLS failure already rules out DNS/connectivity

Say this explicitly — it shows you understand the layering, not just each layer in isolation

Reviewed

§3

Autoscaling

4 Q&A

HPA and Cluster Autoscaler internals, not just the names.

01

Explain how HPA actually works internally, not just 'it scales pods.'

›

HPA polls metrics (metrics-server for CPU/memory, or a custom/external adapter for anything else) on an interval, compares the current value against your target, and computes desired replicas as roughly currentReplicas × (currentMetric / targetMetric), rounding up. It won't scale below minReplicas or above maxReplicas, and has stabilization windows to avoid thrashing — scale-up reacts fast by default, scale-down deliberately waits longer.

The actual formula: desiredReplicas ≈ currentReplicas × (currentMetric/targetMetric) — stating this is a concrete depth signal

metrics-server for CPU/memory; a custom/external adapter for anything else (queue depth, RPS)

Scale-up is fast, scale-down is deliberately slower — asymmetric on purpose, to avoid flapping

Reviewed

02

Pods are Pending due to insufficient resources, and HPA wants to scale further.

›

HPA will happily create more pod objects even with no cluster capacity to schedule them — HPA decides desired replica count, not whether the cluster can host it, so you get a growing pile of Pending pods. Check kubectl describe pod for the exact scheduler reason, and check whether Cluster Autoscaler is enabled and has room to add nodes — a node group at its own max size can't add capacity no matter how badly needed.

HPA and cluster capacity are separate concerns — HPA doesn't know if the cluster can host what it asks for

kubectl describe pod Events gives the exact scheduler reason — don't guess

Cluster Autoscaler at its own max node count is a real, common reason autoscaling 'isn't working' when HPA is functioning correctly

Reviewed

03

How does Cluster Autoscaler decide to add/remove nodes, and how does it interact with HPA?

›

Cluster Autoscaler watches for Pending pods that can't schedule due to insufficient node resources and adds nodes; it watches for underutilized nodes whose pods could be rescheduled elsewhere and removes them after a grace period, if that wouldn't violate a PodDisruptionBudget. HPA decides how many pods should exist; Cluster Autoscaler decides how many nodes are needed to host whatever pods currently exist or are Pending.

Clean division: HPA decides pod count, Cluster Autoscaler decides node count to host them

Node removal respects PodDisruptionBudgets — won't scale down in a way that violates one

Cluster Autoscaler is reactive to Pending pods, not predictive — real lag exists between demand and new node availability

Reviewed

04

What resource requests/limits mistakes actually break autoscaling in production?

›

No requests set means HPA has nothing to calculate a percentage against, since the target is relative to the request — HPA silently can't function. Requests set far below real usage make the scheduler overpack nodes, causing throttling even while HPA thinks pod-level metrics look fine. Limits too close to requests cause throttling/OOMKill during a legitimate spike HPA hasn't caught up to scaling for yet.

No requests defined breaks HPA silently, not loudly — no baseline to compute a percentage against

Requests too low = node overpacking, throttling despite HPA reporting healthy pod-level metrics

Limits too tight against requests = a real spike causes throttling before HPA can react — a timing gap, not a config bug

Reviewed

§4

Helm

3 Q&A

Templates, upgrade failures, secrets handling.

01

What are Helm templates actually doing, versus raw Kubernetes YAML?

›

Helm templates are Kubernetes YAML with Go templating layered in, letting one chart produce different rendered manifests per environment by substituting values from values.yaml — the same chart deploys dev, staging, and prod with different replica counts or image tags, instead of maintaining near-duplicate YAML by hand. helm template renders the final YAML locally without applying it — the right way to review exactly what will deploy before running install/upgrade for real.

One chart, many environments, via values substitution — the actual value proposition over raw YAML

helm template for a dry-run render to review — use before install/upgrade, not after something breaks

This also answers 'why Helm over Kustomize' if asked — templating/parameterization vs patch-based overlays

Reviewed

02

A Helm upgrade fails partway through — what's your process?

›

Check helm status release-name and helm history release-name first to see current state and actual revision. If legitimately broken, helm rollback release-name previous-revision-number reverts the whole release cleanly. Helm 3's --atomic flag on install/upgrade automatically rolls back on failure rather than leaving a half-applied release — the preventive fix, not just the reactive one.

helm status + helm history before doing anything — confirm current state first

helm rollback to a specific prior revision number is the recovery action

--atomic is the preventive answer — auto-rollback on failure, worth naming unprompted

Reviewed

03

How do you handle secrets in Helm charts without committing them to Git?

›

Never put real secret values in values.yaml committed to Git. Common patterns: reference an already-existing Secret created out-of-band via External Secrets Operator pulling from Vault/AWS Secrets Manager/Azure Key Vault, rather than having Helm create it from a plaintext value, or use Helm Secrets/SOPS to encrypt values files at rest and decrypt only at deploy time.

Never commit real secret values in values.yaml, even 'temporarily'

External Secrets Operator (pulling from Vault/cloud secret manager) is the GitOps-clean answer

Helm Secrets/SOPS is the alternative pattern if there's no external secrets operator in place

Reviewed

§5

Docker

4 Q&A

Multi-stage builds and image lifecycle, stated with confidence.

01

Explain a multi-stage Dockerfile and why you'd use one — with real confidence this time.

›

A multi-stage build uses multiple FROM statements in one Dockerfile — an early stage has the full build toolchain to actually build the application, and a final stage starts fresh from a minimal base image and copies ONLY the compiled artifact via COPY --from=build-stage. The build tools themselves never reach the final image, making it dramatically smaller and reducing attack surface.

Multiple FROM statements; only the final stage's contents ship — build tools never reach production

COPY --from=build-stage is the specific mechanism — describe it precisely, not just conceptually

Smaller image + reduced attack surface are the two concrete benefits to state

Reviewed

02

Walk through the full image lifecycle from build to running in production.

›

Build (ideally multi-stage) → tag with the Git SHA rather than latest so every deployed image maps to an exact commit → push to an authenticated private registry → optionally scan for vulnerabilities (Trivy/JFrog Xray) before it's allowed to deploy → Kubernetes pulls it by that exact tag → old images eventually get garbage-collected via a retention policy.

Build → SHA-tag → push to authenticated registry → scan → deploy → eventual GC — know the full chain

SHA-based tagging ties directly into rollback — the previous SHA is always redeployable directly

Mentioning image scanning as a stage, unprompted, is a maturity signal

Reviewed

03

Private registry authentication in Kubernetes — how does a pod actually pull from one?

›

A Kubernetes Secret of type docker-registry holds the credentials, referenced by the pod's imagePullSecrets, or attached to the ServiceAccount so every pod inherits it without repeating it per-pod. Without it, or with a wrong credential, you get ImagePullBackOff with an auth-specific error in kubectl describe pod Events — distinct from a wrong-tag ImagePullBackOff.

Secret type docker-registry, referenced via imagePullSecrets on the pod or the ServiceAccount

ServiceAccount-level attachment avoids repeating the pull secret on every pod spec

Read the specific Events message — auth failure and wrong-tag failure both show as ImagePullBackOff but need different fixes

Reviewed

04

How do you talk about a genuine skill gap honestly without undermining yourself?

›

If something is a 6-7 rather than a 9-10, say so plainly and immediately pair it with the concrete thing you DO know cold — 'I've built and debugged multi-stage Dockerfiles in production, tagged and pushed to a private registry, and handled ImagePullBackOff/OOMKilled troubleshooting; where I'm less sharp is BuildKit-level caching internals' is a confident 7, not an apologetic one. The interviewer explicitly said that's fine — the actual mistake is vagueness with nothing concrete attached, not the honest number.

Honest self-rating is fine — the interviewer said so directly — vagueness attached to it is the actual problem

Pair any 'I'm a 7 here' with one concrete thing you've actually done, immediately, same breath

This reframes 'confidence' as specificity, not as inflating the number

Reviewed

§6

SRE Concepts

3 Q&A

SLI/SLO/SLA, error budget, and burn rate — precise, with real numbers.

01

Full forms and precise definitions — SLI, SLO, SLA.

›

SLI (Service Level Indicator) is the actual measured metric — percentage of successful requests, or p99 latency. SLO (Service Level Objective) is your internal target for that SLI, usually tighter than any external promise, as a buffer. SLA (Service Level Agreement) is the external, often contractual commitment to a customer, typically with a defined penalty if missed — usually looser than the internal SLO so you have room to notice and fix a problem before the contract itself is breached.

SLI = the measurement. SLO = your internal target. SLA = the external contractual promise

SLA is deliberately looser than SLO — that gap is your safety margin before a real contractual breach

Getting the full forms exactly right is a basic correctness check — don't fumble it

Reviewed

02

What's an error budget, in concrete terms?

›

If your SLO is 99.9% success over 30 days, your error budget is the remaining 0.1% — the failure you're explicitly allowed without breaching the objective. For a service handling 10 million requests over that window, 0.1% is 10,000 allowed failed requests. It's a shared resource between reliability work and feature velocity: spending it fast means slowing releases to protect reliability; budget left over is explicit permission to ship faster.

Error budget = 100% − SLO, expressed as an actual allowance, not just an abstract percentage

Convert it to a concrete number (10,000 failed requests of 10 million) — this is what makes it land in an interview

It balances reliability against feature velocity — that's the point of having it at all

Reviewed

03

What's burn rate, and why does it matter more than the raw error budget number?

›

Burn rate is how FAST you're consuming the error budget relative to a sustainable pace — a burn rate of 1 exhausts the entire budget exactly at the end of the SLO window; a burn rate of 10 exhausts a month's budget in about 3 days. It's predictive, not just a snapshot: a fast burn rate is worth paging on immediately even with 95% of budget technically remaining, since at that rate it won't stay remaining for long. Multi-window burn-rate alerts (a short window for fast burns plus a longer window to avoid noise) are the standard pattern.

Burn rate 1 = exactly sustainable for the full window; burn rate 10 = the month's budget gone in ~3 days

It's predictive, not a snapshot — that's why it's pageable even when a lot of budget remains

Multi-window burn-rate alerting is the specific standard pattern — name it if asked how you'd alert on this

Reviewed

§7

Production Incident

2 Q&A

Tying it together — the deploy-then-errors scenario, plus probes.

01

Deployment completed, error rate spikes, application is dying — what do you do, right now?

›

Mitigate first, diagnose after: if the timing correlates with the deploy, the fastest safe action is rolling back immediately (kubectl rollout undo or the Blue-Green/canary equivalent) rather than debugging live in a dying system. Once traffic is stable, pull logs and metrics from the failed window: a failed readiness probe blocking traffic to bad pods, a bad config/env var, a DB migration incompatible with still-running old pods, or a resource limit too tight for the new version's actual usage.

Rollback first, root-cause second — same 'roll back first, understand later' principle from your MTTR material, stay consistent

Correlating timing with the deploy is what justifies rolling back immediately rather than investigating first

Readiness probe failure, bad config, incompatible migration, undersized limits — the four usual suspects for 'broke right after deploy'

Reviewed

02

Readiness probe vs liveness probe — the actual difference, and how each affects a rollout.

›

Liveness answers 'is this container alive, or should it be killed and restarted' — failing it restarts the pod. Readiness answers 'is this container ready for traffic right now' — failing it removes the pod from the Service's Endpoints, no traffic routed, no restart. During a rolling update, readiness is what actually enforces zero-downtime: Kubernetes won't route traffic to a new pod, or consider it 'up' for the rollout, until readiness passes.

Liveness = restart if failing. Readiness = remove from traffic if failing, no restart

Readiness is what actually gates a rolling update's pace and zero-downtime property — direct link to the deployment-strategy section

A missing readiness probe means K8s considers a pod 'up' the instant the container starts, not when it's actually ready

Reviewed

Precision pass · the same gap, closed for good